

计算机系统基础 Lab1: DataLab 实验报告

陈仕卿 25300120002

一、测试结果

在实验目录下执行 ./check_ops.py bits.c, 输出 All 19 functions passed operator checks.。先执行 make clean && make all 重新编译, 再执行 ./btest 全部正确 110/110。两张截图如下。

```
zhanlei@DESKTOP-591C7BE:/mnt/d/code/C++/DataLab_25300120002$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 6/8 operators
oddParity: 11/56 operators
rotateRightBits: 2/16 operators
roundEvenPow2: 11/24 operators
midpointTowardFirst: 20/32 operators
isBetweenEitherOrder: 45/48 operators
mul5Sat: 29/30 operators
classifyAdd3: 30/52 operators
floatScaleThreeHalves: 57/60 operators
floatRoundEven: 35/65 operators
float_i2f: 29/40 operators
bitCount: 26/40 operators
bitReverse: 30/34 operators
All 19 functions passed operator checks.
```

```
zhanlei@DESKTOP-591C7BE:/mnt/d/code/C++/DataLab_25300120002$ make clean
make all
./btest
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
Score  Rating  Errors  Function
1      1        0      signMask
2      2        0      bitXor
3      3        0      negativePart
4      4        0      copyByteWithin
4      4        0      logicalShift
4      4        0      swapNibblePairs
4      4        0      secondLowestZeroBit
5      5        0      oddParity
5      5        0      rotateRightBits
5      5        0      roundEvenPow2
5      5        0      midpointTowardFirst
7      7        0      isBetweenEitherOrder
7      7        0      mul5Sat
7      7        0      classifyAdd3
7      7        0      floatScaleThreeHalves
10     10        0      floatRoundEven
10     10        0      float_i2f
10     10        0      bitCount
10     10        0      bitReverse
Total points: 110/110
```

从两张截图可以看到，19 道题的运算符数量都控制在题目上限之内，比如 mul5Sat 是 29/30、isBetweenEitherOrder 是 45/48、floatScaleThreeHalves 是 57/60，并且每道题的 Errors 都是 0，最终得分 110/110 满分。

二、各函数实现思路

P1signMask

把 1 左移 31 位，得到最高位掩码 0x80000000，直接输出。

P2bitXor

框架只允许用 ~ 和 &，所以用德摩根律把异或拆开： x^y 等于 $\sim(\sim(x \& \sim y) \& \sim(\sim x \& y))$ 。 $x \& \sim y$ 挑出 x 是 1 而 y 是 0 的位， $\sim x \& y$ 挑出 x 是 0 而 y 是 1 的位，两部分本来该取或，但这里没有 |，就把它各自取反后再与非、最后整体取反，把或还原出来。

P3negativePart

先求 x 的符号位掩码：x 算术右移 31 位，负数得到全 1，非负得到 0。取相反数用补码公式 $\sim x + 1$ 。把符号掩码和相反数相与，x 为负就输出 -x，否则输出 0。

P4copyByteWithin

先算出 src 和 dst 对应的实际左移量，也就是各自乘 8： $src \ll 3$ 和 $dst \ll 3$ 。把 x 右移 Xsrc 位再与 255，取到 src 那个字节，再左移 Xdst 位搬到目标字节的位置。然后用 $255 \ll Xdst$ 造出目标字节的掩码取反，与 x 相与，把目标字节清空、其他字节保留，最后两者取或就是答案。

P5logicalShift

要求逻辑右移，但机器上的 $\gg$ 是算术右移，负数会补符号位。先照常得到 $x \gg n$ ，再算 x 的符号掩码 $1 \ll 31$ ，把掩码右移 n 位、再左移 1 位，就得到高 n 位全 1 的掩码，取反后与移位结果相与，多补出来的 1 就被清掉了。

P6swapNibblePairs

思路和 P19 一样。先构造 4 位掩码 $wei4 = 0x0F0F0F0F$ ，由 0x0F 依次左移 8 位、16 位再相加凑出来。然后让 x 右移 4 位与掩码相与，取到每个字节的高 4 位并挪到低位；x 与掩码相与再左移 4 位，把低 4 位挪到高位，两者取或就完成了半字节交换。

P7secondLowestZeroBit

先用公式 $\sim x \& (x + 1)$ 得到最低位的 0：x + 1 会把最低的 0 顶成 1、把更低的 1 全部变成 0，再用 $\sim x$ 一与就只剩下那个位置。然后让 x 加上这个最低位 0，等于把这个 0 填成 1，再重复一遍上面的操作，这次得到的就是倒数第二个 0 的 mask。

P8oddParity

设立一个 Xhalf，通过右移得到 x 的另一半，与 x 本身异或。依次取 16 位、8 位、4 位、2 位、1 位重复 5 次，最后所有位的异或和就折叠到最低位上，得到一个数，与 1 相与取出这一位，再取反就是要求的答案。

P9rotateRightBits

先把 n 处理成 31 以内的正整数 $n \& 31$ 。循环右移由两部分拼成：前半部分右移 n 位，后半部分左移 $32 - n$ 位（用 $33 + \sim n$ 得到）。左移那部分要用掩码 $(1 \ll (33 + \sim n)) + \sim 0$ 把高位清零，避免超出 32 位的残留，两部分相加就是答案。

P10roundEvenPow2

只要把 x 加上一个偏移量再把低 n 位清掉就行。偏移量先取 $2^{(n-1)} - 1$ ，也就是半程减 1；再用商的最低位 $x \text{quot} \& 1$ 判断当前商是奇数还是偶数，商为奇数时偏移量要再加 1，这样恰好等于半程的情形会向上进位到偶数商，商为偶数时则停在半程处向下取整。加完偏移后先右移 n 位再左移 n 位，低位自然清零。

P11midpointTowardFirst

先算出 x 和 y 向下取整的平均值，用 $(x \& y) + ((x \wedge y) \gg 1)$ 这个公式来防止溢出。然后要处理 $x + y$ 为奇数时的平局：这时精确中点落在两个整数之间，要求取离 x 更近的那个。于是取 $x \wedge y$ 的最低位判断 $x + y$ 是否为奇数，再判断 x 和 y 谁大。不能用比较运算符，就先取两者的符号位，异号时 $x < y$ 等价于 x 为负，同号时用 $x - y$ （写成 $x + \sim y + 1$ ）的最高位来判断，综合起来得到标志位 XYs 。最后用 $\sim XYs \& XYj$ 算出偏移量是 0 还是 1，加到中点上去。

P12isBetweenEitherOrder

先取 a 、 b 、 x 的符号位，再算 AX 、 BX 是否同号。因为三数可能横跨正负、直接相减的符号不可靠，所以每个小于判断都按同号、异号分开讨论：异号时只看 x 的符号，同号时用减法的最高位判断。用几个式子把 $x < a$ 、 $x < b$ 、 $a < x$ 、 $b < x$ 分别表达出来，取反就得到大于等于的关系。如果 x 同时不低于 a 且不高于 b ，或者同时不低于 b 且不高于 a ，说明 x 落在区间里，输出 1。

P13mul5Sat

这道题目太难了，试了几十次，终于做出来。先算出 x 的上限和下限： $INT_MAX / 5 = 0x19999999$ ， $INT_MIN / 5 = 0xE6666666$ ，超出这个范围的 x 乘 5 一定溢出。上界用 $(0x19 \ll 24) | (0x99 \ll 16) | (0x99 \ll 8) | 0x99$ 拼出来，下界直接取它的按位反。然后根据 x 的符号和相减结果的符号，算出是否向上或向下溢出，两者都无就说明不溢出。不溢出时才计算 $(x \ll 2) + x$ 作为 $5x$ 输出；溢出就按方向输出 $INT_MAX (\sim MIN)$ 或者 $INT_MIN (1 \ll 31)$ ，三路取或合并。

P14classifyAdd3

把三个数的加法拆成两次和运算，再加上 x 、 y 、 z 和两次和结果的符号位，一共五个符号位。用符号位处理出两次加法各自是否进位：两操作数同号而结果异号就是进位。然后把三个符号位、两个未进位标志和最终结果的符号位一起求和，只有三个加数全为正且两次都没进位时这个和才为正，全为负且两次都进位时才为负，其余情况精确和一定在 int 范围内。最后把和右移 31 位得到 -1 或 0，再和 $!$ 和 1 组合，输出 1、-1 或 0。

P15floatScaleThreeHalves

先把输入的 uf 拆成 s 、 exp 、 $frac$ 三部分。 exp 全是 1 说明是无穷大或者 NaN，直接原样输出； exp 和 $frac$ 都是 0 就是正负零，也原样输出。再把有效数字还原成整数 M ：规格化数取 $(1 \ll 23) | frac$ 、阶码 $exp - 127$ ，非规格化数取 $frac$ 、阶码 -126。乘 $3/2$ 拆成两步：先算 $M3 = M * 3$ ，再整体除以 2 也就是指数减 1。用循环数出 $M3$ 的位宽 p ，结果的指数就是 $E + p - 1$ ，尾数要保留 24 位有效数字。 p 不超过 23 时有效位够用，只需把 $M3$ 右移 1 位再做一次就近取偶。 p 大于 23 时把低的 $p - 23$ 位丢掉，丢弃部分大于半程就进位、恰好等于半程且保留部分最低位是 1 也进位，这就是 round-to-nearest-even。进位后如果保留部分变成 $1 \ll 24$ ，说明多了一位，右移一位并把指数加 1。最后算回阶码，到 $0xFF$ 就是无穷大，否则和符号位、尾数拼回结果。

P16floatRoundEven

先把输入的 uf 拆开变成 s 、 exp 、 $frac$ 三个部分。 exp 全是 1 就是无穷大或者 NaN，原样输出； exp 不小于 150 说明本身就是整数，也原样输出。 exp 小于 126 时绝对值不到 0.5，直接舍入到零：要判断是否为负数（输出 -0），以及是否在 0.5 左侧（输出 0）还是右侧；恰好等于 0.5 的平局按就近取偶进位到 1.0。剩下的情况左移直到它是一个大整数，切掉小数部分，把丢弃部分和半程比

较，判断往哪里进位。进位溢出就向 `exp` 进位，尾数归位；阶码要是变成 `0xFF` 就是无穷大，直接返回。最后数出保留部分的位宽 `p`，阶码取 $127 + p$ ，把小数左移到尾数位置拼回去。

P17float_i2f

其实和上一题差不多，复用了大部分代码。先用循环求出绝对值最高位 1 的位置 `p`，阶码就是 $127 + p$ 。`x` 为 0 直接返回 0；`x` 是 `0x80000000` 时相反数没法用 `int` 表示，直接返回 `0xCF000000`。负数统一取相反数变成正数处理，最后补上符号位。`p` 不超过 23 时有效位不够，把绝对值左移到 23 位宽即可；`p` 大于 23 时低 `p - 23` 位丢不掉，就按丢弃部分和半程的大小决定是否进位，等于半程时看保留部分最低位的奇偶，进位后如果变成 $1 \ll 24$ 就右移一位并把阶码加 1。

P18bitCount

先构建五个掩码，构造方式和 P19 注释里写的一样。掩码用来保留每组的第 0 位。`x` 右移 1 位再和掩码相与，两者相加就是这两位里 1 的总个数。以此类推，依次处理 2 位、4 位、8 位、16 位的分组，最后得到的就是 32 位里面 1 的总个数。

P19bitReverse

先构造出五个掩码：`wei1` 是 1010...1010，`wei2` 是 1100...1100，`wei4` 是 11110000 重复，`wei8` 是 1111111100000000 重复，`wei16` 就是全 1。然后借助掩码运算，先交换相邻 1 位，再相邻 2 位、4 位、8 位，最后交换相邻 16 位，五轮之后 32 位就变成反序了。

三、参考资料

1. CS:APP 第三版第二章，补码、位运算、移位与 IEEE754 单精度格式。
2. CMU 原版 DataLab
3. 作业仓库中的 README.md 与 bits.c 每个函数上方的规则注释。